



Concurrent and Distributed Systems

COS6012-B

UB Number - 25029204

Coursework 2

Department of Computer Science
University of Bradford

1. Distributed vote counting

The distributed voting system was designed using a simple centralised architecture. The idea is that each voting centre works independently, counts its own votes, and then sends the result to a central server. The server is responsible for collecting all these partial results and combining them to produce the final outcome of the election.

This approach is simple to implement and transparent to understand. Each voting centre is implemented as a client. It generates a random number of votes for a small set of predefined parties and sends the result to the server. This simulates how different physical voting locations would report their results. After sending the data, the centre waits for a response from the server to confirm whether the message was accepted.

The central server listens for incoming connections using a `ServerSocket`. Since multiple centres may send their results at the same time, the server handles each connection in a separate thread. This allows the system to process multiple voting centres concurrently, rather than one at a time.

One important issue in this type of system is concurrency. Because several threads may try to update the global vote totals at the same time, there is a risk of race conditions, which could lead to incorrect results (Friesen, 2015). To prevent this, I used a synchronized block when updating the shared data structure that stores the vote totals. This ensures that only one thread can modify the totals at any given moment.

For communication, I decided to use a very simple text-based protocol. Each message follows a format like:

C1 local results: {PartyA=69, PartyB=140, PartyC=125}

Additionally, a validation is also included. If a message has an invalid format or contains negative vote values, the server rejects it and returns an error response. Valid messages are acknowledged with a simple "ACK".

Overall, this design works well for a small-scale simulation. It demonstrates the key ideas of distributed systems, such as independent nodes, communication over a network, and concurrent processing, while still being simple enough to implement within the scope of this assignment.

However, the system has some limitations that can be addressed with architectural improvements. The reliance on a single central coordinator creates a single point of failure, which could be mitigated by introducing a backup coordinator or a more decentralised approach to task distribution. Additionally, the system does not handle failures robustly; this could be improved by implementing timeouts, retry mechanisms, and heartbeat signals to detect and recover from worker or network issues. Finally, the lack of data persistence means that all results are lost after execution, which could be resolved by storing task inputs

and outputs in files or a database, enabling recovery, analysis, and improved system reliability. (Payal and Garg, 2015)

Output example:

```
>> java -cp Coursework2\target\classes uk.ac.bradford.cds.cw2.MainSimulation
>>
CentralAggregatorServer running on port 59010
C4 local results: {PartyA=135, PartyB=65, PartyC=54}
C2 local results: {PartyA=119, PartyB=115, PartyC=146}
C1 local results: {PartyA=130, PartyB=80, PartyC=64}
C3 local results: {PartyA=99, PartyB=183, PartyC=175}
C3 server response: ACK
C2 server response: ACK
C4 server response: ACK
C1 server response: ACK

Final totals:
PartyA = 483
PartyB = 443
PartyC = 439
Winner: PartyA
```

The output shows how each voting centre independently generates its local results and sends them to the central server. Each centre prints its own vote distribution for the parties, followed by a confirmation message (ACK) from the server. This indicates that the data was successfully received and validated. The fact that responses arrive in a slightly mixed order reflects the concurrent nature of the system, where multiple centres communicate with the server at the same time.

After all centres have submitted their results, the server aggregates the votes and prints the final totals for each party, along with the overall winner. This demonstrates that the central server correctly combines all partial results into a single outcome. The final values confirm that the system is working as expected, ensuring consistency despite concurrent updates through the use of synchronized access to shared data.

Files structure and logic functionality:

- a. **MainSimulation.java**: Starts the whole demo, launches the central server and multiple voting centres, waits for them, then prints final results and stops the server.
- b. **CentralAggregatorServer.java**: Runs the TCP server, accepts client connections, aggregates votes safely across threads, and computes the winner.
- c. **ClientHandler.java**: Handles one client connection, parses the incoming vote message, validates it, updates totals on the server, and replies with ACK or ERROR.
- d. **VotingCentreNode.java**: Simulates one voting centre, generates local random votes, sends them to the server, and prints the server response.

2. Distributed numerical integration

The system was designed using a simple distributed architecture based on a coordinator and two worker nodes. The main idea is that the same integration task is sent to two different workers, each applying a different numerical method. This allows the results to be computed in parallel and then compared.

The coordinator node is responsible for interacting with the user. It collects the input, including the function to integrate, the interval $[a, b]$, and the number of subintervals n . Once the input is validated, the coordinator builds a single task and sends it to both workers. Each worker receives the same task but processes it using a different numerical method.

The two worker nodes are independent and run on separate ports. One worker applies the trapezoidal rule, while the other uses the midpoint rule. This separation makes the parallelism explicit, as both computations happen at the same time but in different processes.

Communication between the coordinator and the workers is done using TCP sockets. Each task is sent as a single line of text, and each worker responds with its result and execution time (Payal and Garg, 2015). Additionally, parallelism is achieved by having the coordinator send requests to both workers at the same time using separate threads. This allows both numerical methods to run concurrently, reducing the total execution time compared to running them sequentially (Coulouris et al., 2012).

Another important concern in this design is ensuring that both results are handled correctly, even if one worker fails. For this reason, if a worker returns an error or fails to respond, the coordinator detects this and avoids comparing incomplete results.

Worker Trapezoidal

```
PS C:\Users\Inteli\Documents\Concurrent-and-Distributed-Systems> cd .\Coursework2\  
PS C:\Users\Inteli\Documents\Concurrent-and-Distributed-Systems\Coursework2> javac -d target/integrati  
on-classes src/main/java/uk/ac/bradford/cds/cw2/integration/*.java  
PS C:\Users\Inteli\Documents\Concurrent-and-Distributed-Systems\Coursework2> java -cp target/integrati  
on-classes uk.ac.bradford.cds.cw2.integration.WorkerNode TRAPEZOIDAL 6001  
>>  
Worker TRAPEZOIDAL listening on port 6001  
█
```

Worker Midpoint

```
PS C:\Users\Inteli\Documents\Concurrent-and-Distributed-Systems> cd .\Coursework2\  
PS C:\Users\Inteli\Documents\Concurrent-and-Distributed-Systems\Coursework2> java -cp target/integrati  
on-classes uk.ac.bradford.cds.cw2.integration.WorkerNode MIDPOINT 6002  
>>  
Worker MIDPOINT listening on port 6002  
█
```

Coordinator Node

a. Experiment 1

PREDEF
SIN
a = 0
b = 3.14
n = 10000

```
PS C:\Users\Inteli\Documents\Concurrent-and-Distributed-Systems\Coursework2> java -cp target/integrati
on-classes uk.ac.bradford.cds.cw2.integration.CoordinatorNode
Function type? Enter PREDEF or POLY:
PREDEF
Choose predefined function: SIN, COS, or EXP
SIN
Enter a:
0
Enter b:
3.14
Enter number of subintervals n:
10000

Integration input:
Function type: PREDEF
Function data: SIN
Interval: [0.0, 3.14]
Subintervals n: 10000

TRAPEZOIDAL -> value=1.9999987152948873, timeMs=0
MIDPOINT -> value=1.9999987399438666, timeMs=0

Absolute difference between methods: 2.464897930209986E-8
Faster method: TRAPEZOIDAL
PS C:\Users\Inteli\Documents\Concurrent-and-Distributed-Systems\Coursework2>
```

This experiment integrates the function $\sin(x)$ over the interval $[0, 3.14]$, which approximates the interval $[0, \pi]$.

The expected result of this integral is 2, and both methods produced values very close to this. The small difference observed is due to numerical approximation.

b. Experiment 2

PREDEF
EXP
a = 0
b = 2
n = 20000

```
PS C:\Users\Inteli\Documents\Concurrent-and-Distributed-Systems\Coursework2> java -cp target/integrati
on-classes uk.ac.bradford.cds.cw2.integration.CoordinatorNode
Function type? Enter PREDEF or POLY:
PREDEF
Choose predefined function: SIN, COS, or EXP
EXP
Enter a:
0
Enter b:
2
Enter number of subintervals n:
20000

Integration input:
Function type: PREDEF
Function data: EXP
Interval: [0.0, 2.0]
Subintervals n: 20000

TRAPEZOIDAL -> value=6.389056104254889, timeMs=1
MIDPOINT -> value=6.389056096268564, timeMs=0

Absolute difference between methods: 7.986325378794845E-9
Faster method: MIDPOINT
```

In this experiment, the function e^x was integrated over the interval $[0, 2]$ using 20,000 subintervals.

Both methods produced almost identical results, with only a very small difference between them. This shows that both numerical methods perform well even for exponential functions.

c. Experiment 3

POLY
1,0,0
a = 0
b = 5
n = 10000

```
PS C:\Users\Intelli\Documents\Concurrent-and-Distributed-Systems\Coursework2> java -cp target/integration-classes uk.ac.bradford.cds.cw2.integration.CoordinatorNode
Function type? Enter PREDEF or POLY:
POLY
Enter polynomial coefficients (highest degree first), comma separated:
1,0,0
Enter a:
0
Enter b:
5
Enter number of subintervals n:
10000

Integration input:
Function type: POLY
Function data: 1,0,0
Interval: [0.0, 5.0]
Subintervals n: 10000

TRAPEZOIDAL -> value=41.666666875000026, timeMs=1
MIDPOINT -> value=41.666666562499955, timeMs=0

Absolute difference between methods: 3.125000702652869E-7
Faster method: MIDPOINT
```

This experiment uses a polynomial function x^2 over the interval $[0, 5]$ with 10,000 subintervals.

Both numerical methods (Trapezoidal and Midpoint) produced results very close to the expected analytical value (approximately 41.67). The difference between the two methods is extremely small, showing that both approaches are accurate for this function.

The execution time for both methods is very low, as the computation is relatively simple.

Files structure and logic functionality:

- CoordinatorNode.java**: Collects user input, validates it, sends the same integration task to two workers in parallel, receives both results, and prints comparison (difference and faster method).
- MathUtils.java**: Runs a worker server on a port, receives tasks from the coordinator, validates/parses the message, computes the integral using its assigned method (TRAPEZOIDAL or MIDPOINT), and returns OK or ERR.
- WorkerNode.java**: Shared math utility class that validates function definitions, builds the function (SIN, COS, EXP, or polynomial), and implements trapezoidal and midpoint numerical integration.

3. Load balancing

a. Review

Load balancing is essential in distributed systems because it improves overall performance by distributing incoming tasks or requests across multiple servers or nodes (Kaur and Kaur, 2015). Instead of overloading a single machine, the workload is shared, reducing response time and increasing throughput. This leads to more efficient resource utilization, as all available nodes contribute to processing rather than leaving some idle while others are overwhelmed. (Coulouris et al., 2012)

In terms of scalability, load balancing enables systems to grow efficiently by adding more nodes (horizontal scaling). As demand increases, new workers or servers can be integrated without major architectural changes. This flexibility is critical in real-world systems where workloads are unpredictable.

Additionally, load balancing improves availability and fault tolerance by ensuring that if one node fails, requests can be redirected to others, preventing total system failure. From a security perspective, load balancers can act as a protective layer by hiding backend servers, mitigating Distributed Denial of Service (DDoS) attacks, and filtering malicious traffic before it reaches internal components.

b. Design and implementation

The load balancing system was designed using a simple distributed architecture composed of three main components: clients, a load balancer, and multiple servers. The main goal of this design is to simulate how incoming requests can be distributed across different servers to improve performance and resource usage.

The clients represent independent sources of requests. In this implementation, they are simulated as threads that send a single request to the load balancer. Each request contains a unique identifier, which allows it to be tracked throughout the system.

The load balancer acts as the central component. It listens for incoming client connections and is responsible for deciding which server should handle each request. To keep the system simple and easy to explain, the Round Robin algorithm was used. This means that each new request is assigned to the next server in a fixed order, cycling through all available servers.

The servers are independent nodes that process the requests assigned by the load balancer. Each server listens on its own port and handles one request at a time. When a request is received, the server records the start time, simulates some processing using a short delay, and then records the end time. These timestamps are sent back to the load balancer along with the server identifier.

Communication between all components is done using TCP sockets with a text-based protocol. The load balancer forwards client requests to the selected server and then receives the processing result before sending a confirmation back to the client.

One important aspect of this design is concurrency. Multiple clients can send requests at the same time, so the load balancer must handle concurrent connections. This is achieved by creating a new thread for each incoming client request. As a result, requests may arrive in a different order than they were created, which explains why the Round Robin distribution is not always perfectly sequential in the output.

The system also records important timing information for each request, including the time it was received by the load balancer, the server that processed it, and the start and end processing times. This information is printed in the console and is used to demonstrate how the load balancing algorithm distributes the workload.

This design has several advantages. It is simple to implement, clearly demonstrates the concept of load balancing, and allows multiple requests to be handled concurrently. It also makes it easy to observe how the Round Robin algorithm distributes requests across servers.

However, the design also has some limitations. The load balancer is a single point of control, and there is no mechanism for handling server failures or dynamically adjusting the number of servers. The communication protocol is also very basic and does not include retries or fault tolerance.

Load balancing in execution:

```
PS C:\Users\Intel\Documents\Concurrent-and-Distributed-Systems\Coursework2> java -cp target/integration-classes uk.ac.bradford.cds.cw2.loadbalancer.SimulationMain
>>
Load balancer listening on port 5000
Server S2 listening on port 6002
Server S3 listening on port 6003
Server S1 listening on port 6001
requestId=C2, timeReceived=1775826172866, serverId=S1, startTime=1775826172873, endTime=1775826173142
Client C2 got: OK|C2
requestId=C7, timeReceived=1775826172867, serverId=S2, startTime=1775826172873, endTime=1775826173202
Client C7 got: OK|C7
requestId=C8, timeReceived=1775826172866, serverId=S2, startTime=1775826172873, endTime=1775826173297
Client C8 got: OK|C8
requestId=C5, timeReceived=1775826172866, serverId=S1, startTime=1775826172873, endTime=1775826173302
Client C5 got: OK|C5
requestId=C1, timeReceived=1775826172866, serverId=S3, startTime=1775826172873, endTime=1775826173318
Client C1 got: OK|C1
requestId=C3, timeReceived=1775826172866, serverId=S2, startTime=1775826172873, endTime=1775826173391
Client C3 got: OK|C3
requestId=C4, timeReceived=1775826172866, serverId=S3, startTime=1775826172874, endTime=1775826173433
Client C4 got: OK|C4
requestId=C6, timeReceived=1775826172866, serverId=S1, startTime=1775826172874, endTime=1775826173434
Client C6 got: OK|C6
All clients finished. Load balancer and servers are still running.
```

The output shows that client requests are being distributed across multiple servers (S1, S2, and S3), confirming that the load balancer is working correctly. Each request is received, assigned to a server, and processed with recorded start and end times, which demonstrates how the workload is shared across the system.

Although the distribution is not perfectly sequential, this is expected due to concurrent processing. Since multiple requests are handled at the same time, the order may vary. Overall, the results show that the Round Robin algorithm is successfully distributing requests and enabling parallel processing.

Files structure and logic functionality:

- c. **SimulationMain.java**: Starts three backend servers and one load balancer, then spawns client requests to test the distribution flow.
- d. **LoadBalancerNode.java**: Accepts client requests, validates them, assigns each request to a backend server using round-robin, forwards the request, validates server replies, logs timing data, and returns a final response to the client.
- e. **ServerNode.java**: Receives forwarded requests from the load balancer, validates message format, simulates processing time, and replies with completion data (DONE|...).

4. References

Coulouris, G.F., Dollimore, J., Kindberg, T. and Blair, G. (2012). *Distributed systems : concepts and design*. Boston: Addison-Wesley.

Friesen, J. (2015). *Java threads and the concurrency utilities*. New York: Apress.

Kaur, S. and Kaur, G. (2015). A Review of Load Balancing Strategies for Distributed Systems. *International Journal of Computer Applications*, 121(18), pp.45–47.
doi:<https://doi.org/10.5120/21644-4985>.

Kurose, J.F. and Ross, K.W. (2007). *Study companion : computer networking : a top-down approach featuring the Internet*. Boston: Addison-Wesley.

Payal, P. and Garg, A. (2015). A Framework to Optimize Load Balancing to Improve the Performance of Distributed Systems. *International Journal of Computer Applications*, 122(15), pp.24–27. doi:<https://doi.org/10.5120/21777-5050>.